FIAP ON

BUILDING STATISTICAL MODELS

TÉCNICAS DE ENSEMBLE LEARNING

3

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaquealmeida@outlook.com

LISTA DE FIGURAS

Figura 1 – Exemplo de *Ensembling Baggin* e *Boosting*.....5
Figura 2 – Diagrama de funcionamento de *Bootstrap*.....6
Figura 3 – *Bagging* implementado no algoritmo RandomForest 7
Figura 4 – Exemplo de execução do RandomForest com e sem *Bootstrap*.....8
Figura 5 – Exemplo simplificado de *Boosting* aplicado a árvores de decisão 10
Figura 6 – *Comparation* entre *Bagging* e *Boosting*.....10
Figura 7 – Comparação do tempo de processamento do XGBoost 11
Figura 8 – Comparação da adoção de algoritmos em competições de ML.....12
Figura 9 – Comparação do tempo de processamento entre CatBoost, XGBoost e LightGBM 13
Figura 10 – Linha do tempo do lançamento 14
Figura 11 – Comparação da função perda para XGBoost, LightGBM e CatBoost....14
Figura 12 – Preparação do ambiente para execução dos algoritmos de *boosting*....15
Figura 13 – Execução dos 4 algoritmos e comparação entre eles.....16
Figura 14 – Execução dos 4 algoritmos e comparação entre eles, com aumento de base 17
Figura 15 – Execução dos 4 algoritmos e comparação entre eles, com ajustes de parâmetros no CatBoost 18
Figura 16 – Implementação do lação para avaliar diferentes distribuições de bases treino-teste 19
Figura 17 – Resultados das 5 iterações do lação, com diferentes bases treino-teste 19

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

SUMÁRIO

1 TÉCNICAS DE ENSEMBLE LEARNING.....4
2 *BAGGING*.....6
3 *BOOSTING* 9
4 MODERNOS ALGORITMOS DE *BOOSTING*.....11
5 ALGORITMOS EM AÇÃO (OU COMO APLICAR NO *PYTHON*).....15
REFERÊNCIAS.....20

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1 TÉCNICAS DE ENSEMBLE LEARNING

Com a evolução natural do campo de pesquisa e estudo de *machine learning*, tivemos o advento de inúmeras novas técnicas, muitas das quais possíveis dada também à evolução da capacidade de processamento.

Chamamos de *ensemble methods* aquelas técnicas (implementadas através de algoritmos) que se utilizam de diversos diferentes algoritmos ou métodos que não somente um único método "puro", por assim dizer. Poderíamos classificar um algoritmo de regressão simples como um método "puro" ou ainda árvores de probabilidades de decisão.

Um algoritmo já visto em capítulos anteriores e que pode ser classificado como um algoritmo *ensemble* é o Random Forest. Na composição desse algoritmo há técnicas de:

- Árvores de decisão.
- Bagging.
- Entre outras.

As técnicas de *bagging*, por si só, são uma implementação interessante nos modernos algoritmos de *machine learning*, pois têm demonstrado bastante eficácia com um esforço menor de codificação por parte do analista. Assim como é possível utilizar *bagging* de forma "automática" quando utilizamos o RandomForest, também podemos utilizar esta técnica em conjunto com outros algoritmos "puros", como árvores de decisão[1].

Outra técnica genérica que pode ser aplicada a diversos algoritmos tanto de regressão quanto de classificação é a chamada *boosting*. Existem diversos algoritmos que implementam esta técnica, por exemplo:

- AdaBoost.
- Extreme Gradient Boosting.
- LigthGBM – Ligth Gradient Boosting Machine.

[1] A implementação que faremos a seguir, na prática, implementa parte do RandomForest.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

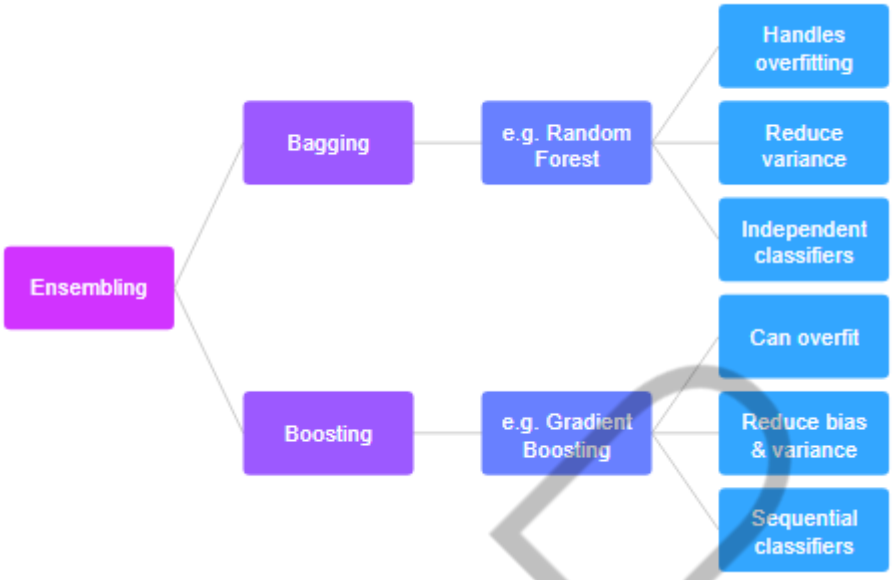


Figura 1 – Exemplo de *Ensembling Baggin* e *Boosting*
Fonte: https://medium.com/@priyankur.sarkar/an-intro-to-ensemble-learning-in-machine-learning-5ed8792af72d

Temos visto recentemente uma evolução significativa em performance de algoritmos *ensemble*, com atenção especial para:

- *Extreme Gradient Boosting* (XGBoost): versão inicial lançada em 2014, ganhou bastante popularidade por ter sido utilizado com sucesso por vencedores de competições de *machine learning*.
- LigthGBM: desenvolvido inicialmente pela Microsoft e lançado em 2016, a abordagem deste algoritmo é parecida com a do XGBoost, porém o custo computacional de processamento é relativamente menor (vários *benchmarkings* mostram uma ordem de cerca de 30% do tempo de processamento do XGBoost) com performance parecida ou em muitos casos até melhor.
- CatBoost: lançado em 2017, o CatBoost é mais um algoritmo da família dos "MLBoosters" que também tem custo menor de processamento computacional e bons resultados.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2 *BAGGING*

O nome *bagging*, diferentemente do que a tradução[2] possa sugerir, na verdade é um acrônimo de *Bootstrap AGGregation*. *Bootstrap*, resumidamente, é uma técnica de geração de várias amostras a partir de um conjunto de dados inicial. Isso pode ser utilizado, por exemplo, ao invés de trabalharmos com testes de hipóteses tradicionais (onde necessitamos gerar novos cenários de comparação).

Essa técnica foi proposta inicialmente em 1979, por Brad Efron, demonstrando que a partir do conjunto de amostras era possível calcular uma série de estatísticas para análise dos dados.

Podemos ver na Figura "Diagrama de funcionamento de *Bootstrap*" um diagrama que demonstra o funcionamento do *Bootstrap*.

Figura 2 – Diagrama de funcionamento de *Bootstrap*
Fonte: https://blogs.sas.com/content/iml/2018/12/12/essential-guide-bootstrapping-sas.html

Os algoritmos que implementaram essa técnica como parte da etapa de construção de modelos demonstraram alguns benefícios:

- Redução de variância dos modelos.
- Impacto direto na prevenção de overfit.

[2] Bag em inglês significa sacola.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- Como consequente, aumento da acurácia.

Um dos algoritmos que tiveram grande notoriedade foi o *Random Forest*, devido aos resultados de melhoria de performance expressivos. Basicamente, a ideia é a geração da floresta, ou seja, do conjunto de árvores, cada qual a partir de uma amostra gerada por *bootstrap*.

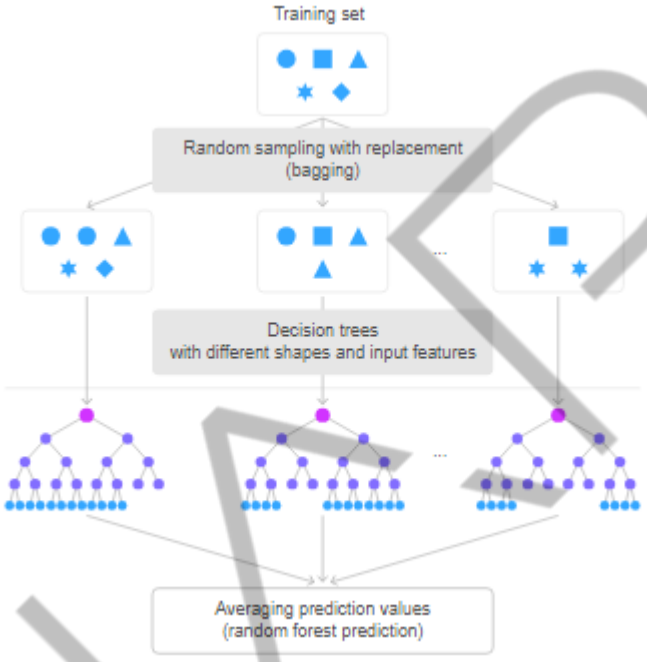


Figura 3 – *Bagging* implementado no algoritmo RandomForest
Fonte:
https://www.researchgate.net/publication/351175730_Ensemble_Learning_Approach_for_the_Prediction_of_Quantitative_Rock_Damage_Using_Various_Acoustic_Emission_Parameters

O algoritmo Random Forest permite que façamos a escolha se queremos utilizar amostras geradas por *bootstrap* para construção das árvores ou não. A opção de utilização de *bootstrap* é padrão, ou seja, se nada definirmos diferente, será utilizado. Veremos um exemplo com uso e sem o uso dessa técnica.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
[25] import pandas as pd
     from sklearn.ensemble import RandomForestClassifier
     from sklearn.model_selection import train_test_split
     from sklearn.metrics import accuracy_score

df = pd.read_csv("https://raw.githubusercontent.com/diogenesjusto/FIAP-On-Line-Graduation/main/datasets/arvore_titanic.csv")
du_Sex = pd.get_dummies(df['Sex'])
du_Pclass = pd.get_dummies(df['Pclass'], prefix='class')
dtAge = df['Age'].fillna(df.groupby('Pclass')['Fare'].transform('mean'))

df = df.join(du_Sex)
df = df.join(du_Pclass)
df['AgeFilledNa'] =dtAge

for i in range(10):
  x_treino, x_teste, y_treino, y_teste = train_test_split(df[['female', 'class_1','class_2', 'SibSp', 'Parch', 'AgeFilledNa']]
                                                          , df['Survived'],test_size=0.3266, random_state=i)

  # Bootstrap Ativo
  mod = RandomForestClassifier(n_estimators = 500, random_state = 1, bootstrap=True)
  mod = mod.fit(x_treino, y_treino)
  y_prev = mod.predict(x_teste)
  accuracy_bootstrap_true = accuracy_score(y_teste, y_prev)

  # Bootstrap Desativado
  mod = RandomForestClassifier(n_estimators = 500, random_state = 1, bootstrap=False)
  mod = mod.fit(x_treino, y_treino)
  y_prev = mod.predict(x_teste)
  accuracy_bootstrap_false = accuracy_score(y_teste, y_prev)

  print(accuracy_bootstrap_true, accuracy_bootstrap_false, accuracy_bootstrap_true-accuracy_bootstrap_false)

0.791095890410959 0.7876712328767124 0.003424657534246589
0.7671232876712328 0.7465753424657534 0.020547945205479423
0.8013698630136986 0.7671232876712328 0.03424657534246578
0.8047945205479452 0.8013698630136986 0.003424657534246589
0.815068493150685 0.8116438356164384 0.003424657534246589
0.8116438356164384 0.8047945205479452 0.006849315068493178
0.8116438356164384 0.8184931506849316 -0.006849315068493178
0.7876712328767124 0.7808219178082192 0.006849315068493178
0.821917808219178 0.8116438356164384 0.010273972602739656
0.8184931506849316 0.8116438356164384 0.006849315068493178
```

Figura 4 – Exemplo de execução do RandomForest com e sem *Bootstrap*
Fonte: Elaborada pelo autor (2022)

Veja, na Figura “Exemplo de execução do RandomForest com e sem *Bootstrap*”, que executamos um laço com 10 repetições variando o parâmetro random_state[3] (=i) na função train_test_split, e sinalizou-se na parte inferior somente 1 resultado em que o não uso do *bootstrap* tenha dado resultado melhor que com o uso. Esse é um exemplo, poderíamos executar mais vezes e com outras bases de dados para validar. Em geral, o uso desse parâmetro mostra um resultado melhor.

[3] Significa que executamos a construção do modelo através do algoritmo Random Forest com diferentes distribuições aleatórias de dados, pois mudou-se a cada interação do lação a composição de dados em treino e teste.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

3 *BOOSTING*

Quando estamos construindo modelos de *machine learning*, testamos inúmeros modelos diferentes na busca de um modelo ideal, ou seja, que minimize a função perda (erro de previsão). É bastante comum encontrar modelos que funcionam bem para determinadas situações e não outras. E a ideia de usar 2 (ou mais) modelos, ao invés de somente 1, passa muitas vezes na cabeça de um analista ou cientista de dados. Esta é a proposta básica do *boosting*[4]: podem vários modelos "fracos" juntos trazerem um resultado melhor que somente um modelo considerado "mais forte"?

Descrito na literatura como problema de hipótese de *boosting*, desde o seu lançamento como hipótese, a aplicação, em diferentes formas de técnicas de *boosting*, tem se mostrado bastante eficiente.

Na Figura "Exemplo simplificado de Boosting aplicado a árvores de decisão", vemos um diagrama um tanto quanto simplificado, mas que permite entender o seu funcionamento:

- Inicialmente, é construída uma árvore de decisão bastante simples, com somente uma variável do tipo binária, separando bem 2 classes.
- Tomando-se como base esta primeira árvore (veja a ideia de que *boosting* é um algoritmo sequencial, diferentemente da técnica de *baggin*, quando as árvores ou, generalizando, os modelos são processados em paralelo), há uma segunda separação (uma segunda árvore) que é independente da primeira, porém ambas, juntas, resolvem melhor o problema que uma árvore somente.
- E assim por diante.

[4] Na verdade, ou melhor, na prática, tanto as técnicas de *bagging* quanto *boosting* funcionam como uma agregação de vários modelos.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

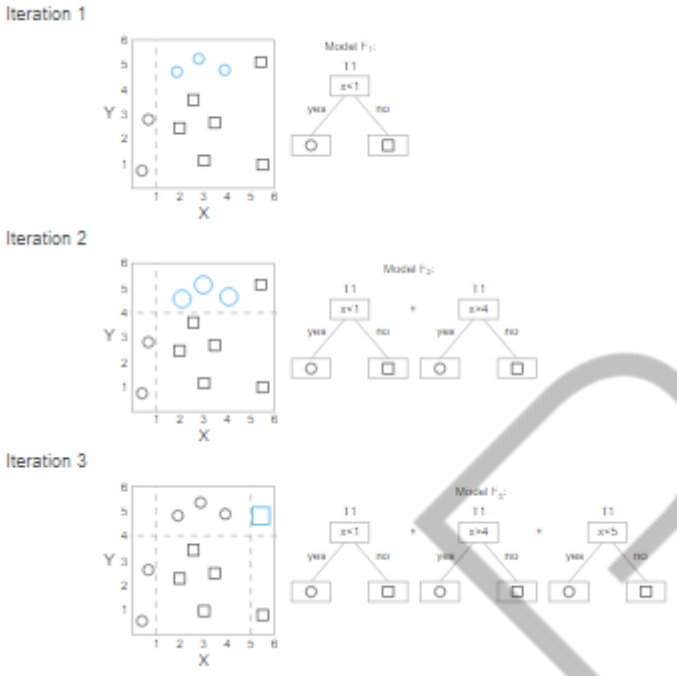


Figura 5 – Exemplo simplificado de *Boosting* aplicado a árvores de decisão
Fonte: Elaborada pelo autor (2022)

Quando comparamos ambas as técnicas de *Bagging* e *Boosting*, no que diz respeito à ordem de execução, como citado anteriormente, podemos realizar uma comparação utilizando-se como base a Figura "*Comparation* entre Bagging e *Boosting*".

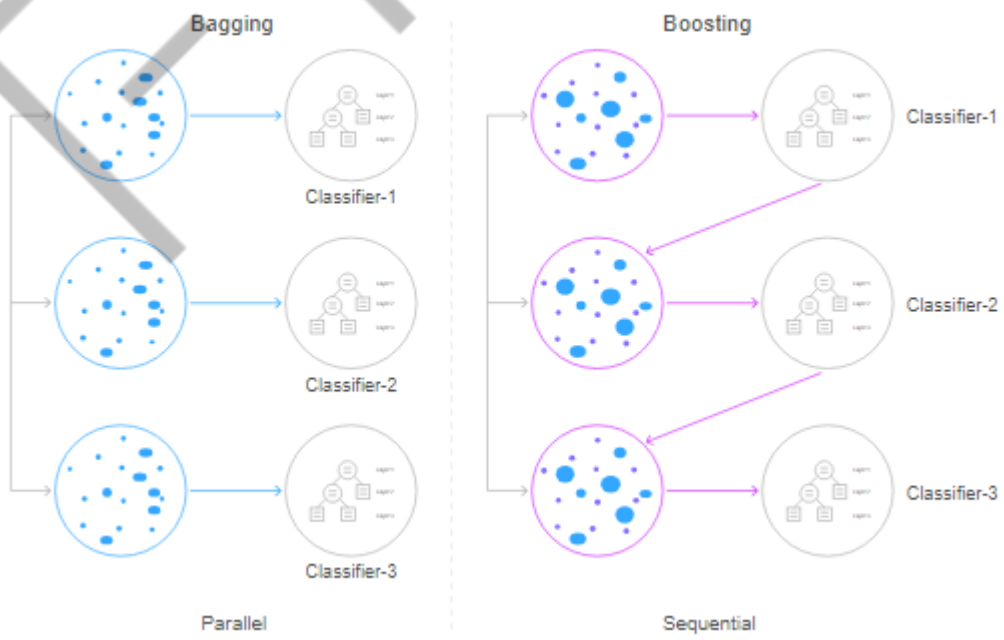


Figura 6 – *Comparation* entre *Bagging* e *Boosting*
Fonte: Elaborada pelo autor (2022)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

4 MODERNOS ALGORITMOS DE *BOOSTING*

O sucesso na implementação de técnicas de *boosting* incitou diversas pesquisas e a criação de novos e poderosos algoritmos. Em 2016, Tianqi Chen apresentou[5], no congresso KDD (SIGKDD) daquele ano, os resultados de sua pesquisa, orientado pelo Prof. Carlos Guestrin, mostrando o algoritmo denominado *Extreme Gradient Boosting (*apelidado de XGBoost).

Rapidamente, esse algoritmo tornou-se muito conhecido pelos diversos relatos de competições de *machine learning*, nos quais os ganhadores relataram seu uso em suas soluções em 2016[6]. Há uma série de razões para seu sucesso, o custo de processamento é uma delas.

Veja, na Figura “Comparação do tempo de processamento do XGBoost”, um gráfico comparativo com o tempo de processamento entre 2 bibliotecas em *Python* e R, em relação ao uso do XGBoost em diferentes máquinas com 1, 2 e 4 cores de processamento.

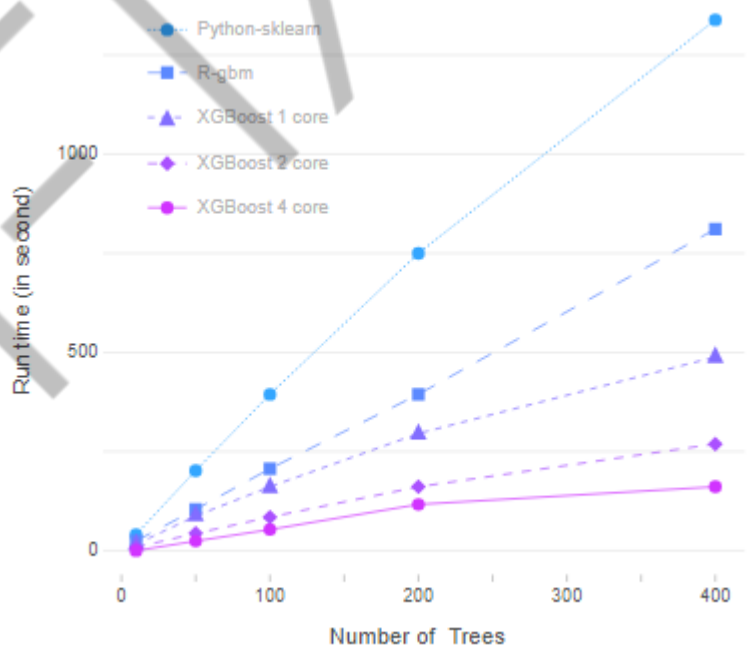


Figura 7 – Comparação do tempo de processamento do XGBoost

[5] Veja a apresentação gravada neste link: https://dl.acm.org/doi/abs/10.1145/2939672.2939785.
[6] Veja um relato breve neste link: https://www.c-sharpcorner.com/article/xgboost-the-choice-of-champions/.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Fonte: https://medium.com/xgboost-all-you-need-to-know/introduction-7c4d8f94bbe1

Um grande *player* de tecnologia (Microsoft), na busca de bem posicionar-se na "corrida" pelas soluções de *machine learning*, apresentou[7], através de seus laboratórios, uma nova proposta de algoritmo de *Boosting*. O LightGBM tornou-se popular rapidamente, também em função de competições de *machine learning*.

Veja, na Figura “Comparação da adoção de algoritmos em competições de ML”, que o algoritmo superou a popularidade do XGBoost em utilização por times em competições.

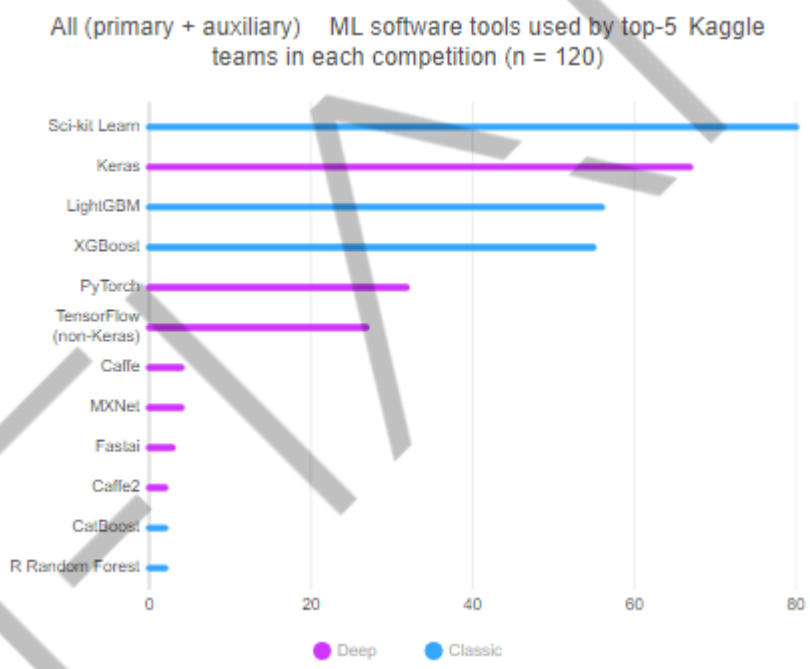


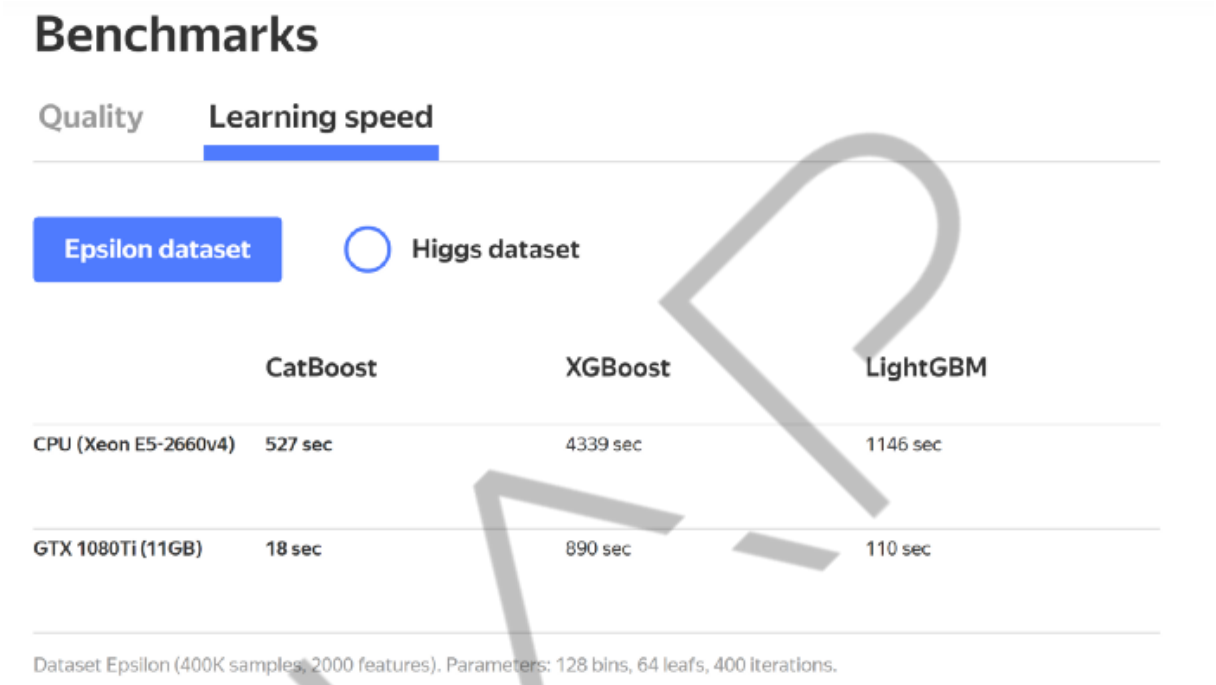
Figura 8 – Comparação da adoção de algoritmos em competições de ML
Fonte: https://keras.io/why_keras/

Na Figura “Comparação da adoção de algoritmos em competições de ML”, há 2 tipos de algoritmos, aqueles utilizados para dados não estruturados (Deep, relativo ao Deep Learning) e estruturados (Classic). Um pouco mais abaixo, aparece outro algoritmo de *boosting*: CatBoost. O "irmão mais novo" dos 3. E o que de melhor um novo algoritmo de *boosting* poderia trazer? Mais *performance* (menor custo computacional).

[7] Veja o *site* da Microsoft Research: https://www.microsoft.com/en-us/research/project/lightgbm/.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Veja, na Figura “Comparação do tempo de processamento entre CatBoost, XGBoost e LightGBM”, o tempo de processamento dos 3 algoritmos em uma determinada situação.



Benchmarks

Quality | Learning speed

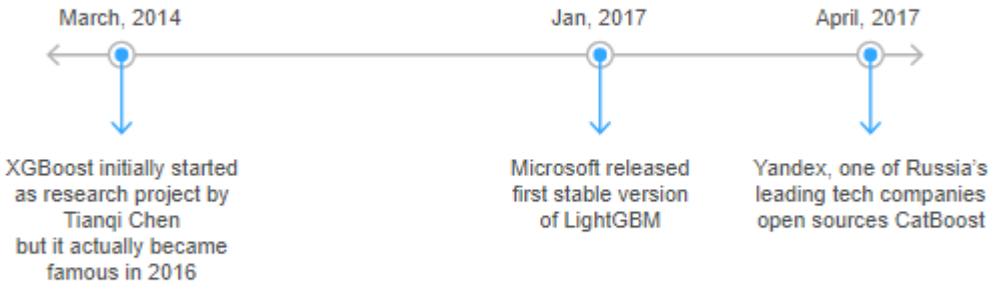
Epsilon dataset | Higgs dataset

	CatBoost	XGBoost	LightGBM
CPU (Xeon E5-2660v4)	527 sec	4339 sec	1146 sec
GTX 1080Ti (11GB)	18 sec	890 sec	110 sec

Dataset Epsilon (400K samples, 2000 features). Parameters: 128 bins, 64 leafs, 400 iterations.

Figura 9 – Comparação do tempo de processamento entre CatBoost, XGBoost e LightGBM
Fonte: https://towardsdatascience.com/catboost-regression-in-6-minutes-3487f3e5b329

Existem diversos artigos comparando a utilização destes 3 algoritmos, em diferentes situações, datasets etc. Veja abaixo, na Figura “Linha do tempo do lançamento”, uma linha do tempo do lançamento deles.



PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Figura 10 – Linha do tempo do lançamento
Fonte: https://www.kdnuggets.com/2018/03/catboost-vs-light-gbm-vs-xgboost.html

Claro que mais importante que o custo computacional (admitindo que o custo seja relativamente parecido, e não se torne algo inviável) é ainda a eficiência ao processar modelos preditivos. Isto é, qual dos algoritmos tem um melhor resultado em termos de minimização da função perda? (ou seja, menor erro de previsão). Não há uma resposta simples para essa pergunta, pois existem variações dependendo desde o tipo de dados, entre inúmeras outras circunstâncias.

Vejamos uma comparação na tabela da Figura “Comparação da função perda para XGBoost, LightGBM e CatBoost”.

	XGBoost	Light BGM		CatBoost	
Parameters Used	max_depth: 50 learning_rate: 0.16 min_child_weight: 1 n_estimators: 200	max_depth: 50 learning_rate: 0.1 num_leaves: 900 n_estimators: 300		depth: 10 learning_rate: 0.15 l2_leaf_reg= 9 iterations: 500 one_hot_max_size = 50	
Training AUC Score	0.999	Without passing indices of categorical features	Passing indices of categorical features	Without passing indices of categorical features	Passing indices of categorical features
		0.992	0.999	0.842	0.887
Test AUC Score	0.789	0.785	0.772	0.752	0.816
Training Time	970 secs	153 secs	326 secs	180 secs	390 secs
Prediction Time	184 secs	40 secs	156 secs	2 secs	14 secs
Parameter Tuning Time (for 81 fits, 200 iteration)	500 minutes	200 minutes		120 minutes	

Figura 11 – Comparação da função perda para XGBoost, LightGBM e CatBoost
Fonte: https://www.kdnuggets.com/2018/03/catboost-vs-light-gbm-vs-xgboost.html

Mesmo utilizando-se a tabela como referência, não é trivial obter uma resposta. A métrica para avaliar o resultado de menor erro em previsão, nessa tabela, é o Test AUC Score, em que a nota mais alta é atribuída a uma parametrização do CatBoost. Mas veja, esse é somente um exemplo, não significa que em todos os casos (ou em seu dataset) isso também ocorrerá.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

5 ALGORITMOS EM AÇÃO (OU COMO APLICAR NO *PYTHON*)

Para iniciarmos a utilização dos principais algoritmos de *boosting*, escolhemos o mesmo dataset de classificação já utilizado neste capítulo. Assim, seguimos com a carga de bibliotecas no Colab, sendo que o CatBoost não está instalado[8] e necessitamos desta tarefa. Veja, na Figura “Preparação do ambiente para execução dos algoritmos de *boosting*”, esta etapa.

```
!pip install catboost

from sklearn.ensemble import GradientBoostingClassifier
from xgboost import XGBClassifier
import lightgbm as lgb
import catboost as ctb
```

```
Looking in indexes: https://pypi.org/simple, https://us-python.pkg.dev/colab-wheels/public/simple/
Collecting catboost
  Downloading catboost-1.0.6-cp37-none-manylinux1_x86_64.whl (76.6 MB)
     |████████████████████████████████| 76.6 MB 1.2 MB/s
Requirement already satisfied: scipy in /usr/local/lib/python3.7/dist-packages (from catboost) (1.7.3)
Requirement already satisfied: six in /usr/local/lib/python3.7/dist-packages (from catboost) (1.15.0)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.7/dist-packages (from catboost) (3
Requirement already satisfied: graphviz in /usr/local/lib/python3.7/dist-packages (from catboost) (0.1
Requirement already satisfied: plotly in /usr/local/lib/python3.7/dist-packages (from catboost) (5.5.0
Requirement already satisfied: pandas>=0.24.0 in /usr/local/lib/python3.7/dist-packages (from catboost
Requirement already satisfied: numpy>=1.16.0 in /usr/local/lib/python3.7/dist-packages (from catboost)
Requirement already satisfied: pytz>=2017.3 in /usr/local/lib/python3.7/dist-packages (from pandas>=0.
Requirement already satisfied: python-dateutil>=2.7.3 in /usr/local/lib/python3.7/dist-packages (from
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.7/dist-packages (from matplotlib
Requirement already satisfied: pyparsing!=2.0.4,!=2.1.2,!=2.1.6,>=2.0.1 in /usr/local/lib/python3.7/di
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.7/dist-packages (from matpl
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.7/dist-packages (from kiwis
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.7/dist-packages (from plotly-
Installing collected packages: catboost
Successfully installed catboost-1.0.6
```

Figura 12 – Preparação do ambiente para execução dos algoritmos de *boosting*
Fonte: Elaborada pelo autor (2022)

Vamos agora estruturar nosso *pipeline* de *machine learning* para execução destes quatro algoritmos:

- GBM: *Gradient Booting* é a versão inicial da implementação desta técnica.
- XGBoost.
- LightGBM.
- CatBoost

Ainda no mesmo código, estruturaremos a separação de bases de treino e teste e posterior laço para execução da construção do modelo, previsão e cálculo da

[8] Ao menos enquanto este texto estava sendo escrito.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

acurácia e tempo de processamento. Veja na Figura “Execução dos 4 algoritmos e comparação entre eles”.

```
from catboost.core import CatBoostClassifier
import time

classifiers = [
    GradientBoostingClassifier(),
    XGBClassifier(),
    lgb.LGBMClassifier(),
    ctb.CatBoostClassifier()
]
classifier_names = ['GBC', 'XGBoost', 'LightGBM', 'CatBoost']

x_treino, x_teste, y_treino, y_teste = train_test_split(df[['female', 'class_1','class_2', 'SibSp', 'Parch', 'AgeFilledNa']]
                                                        , df['Survived'],test_size=0.3266, random_state=1)

for idx, clf in enumerate(classifiers):
  inicio = time.time()

  if classifier_names[idx]=='CatBoost':
    mod = clf.fit(x_treino, y_treino, silent= True)
  else:
    mod = clf.fit(x_treino, y_treino)

  y_prev = mod.predict(x_teste)
  acc = accuracy_score(y_teste, y_prev)
  fim = time.time()

  print(classifier_names[idx], '| Acc=',acc,'| Tempo para execução:', fim-inicio, 'segundos')
```

```
GBC | Acc= 0.7636986301369864 | Tempo para execução: 0.08865189552307129 segundos
XGBoost | Acc= 0.7705479452054794 | Tempo para execução: 0.05170583724975586 segundos
LightGBM | Acc= 0.7808219178082192 | Tempo para execução: 0.040944576263427734 segundos
CatBoost | Acc= 0.7636986301369864 | Tempo para execução: 0.5190300941467285 segundos
```

Figura 13 – Execução dos 4 algoritmos e comparação entre eles
Fonte: Elaborada pelo autor (2022)

Repare que utilizamos uma instrução condicional (if) específica para o CatBoost. Neste caso, esta opção foi para conseguirmos implementar o modo “não verboso”, isto é, não aparecer na janela de código cada iteração do algoritmo[9].

Nota-se que, diferentemente, vimos nos exemplos de artigos anteriores que o CatBoost demorou mais e, neste caso, o LightGBM conseguiu uma acurácia maior. Temos aí uma base de dados relativamente pequena e poderíamos querer realizar esses testes em uma base de dados maior. Isso pode ser simulado multiplicando a base de treino (fazendo cópias idênticas dos mesmos dados).

Vamos realizar essa simulação e ver como os algoritmos se comportam? Veja, na Figura “Execução dos 4 algoritmos e comparação entre eles, com aumento de base", que agora temos uma base de treino de 38.336 linhas; e ainda o CatBoost, que seria o mais rápido, está demorando mais (e atingindo um resultado pior!).

[9] Faça um teste alterando o parâmetro silent=True para False, e veja o modo "não silencioso".

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```python
# 2a tentativa: aumento da base de treino em 10x
from catboost.core import CatBoostClassifier
import time

classifiers = [
    GradientBoostingClassifier(),
    XGBClassifier(),
    lgb.LGBMClassifier(),
    ctb.CatBoostClassifier()
]
classifier_names = ['GBC', 'XGBoost', 'LightGBM', 'CatBoost']

x_treino, x_teste, y_treino, y_teste = train_test_split(df[['female', 'class_1','class_2', 'SibSp', 'Parch', 'AgeFilledNa']]
                                                        , df['Survived'],test_size=0.3266, random_state=1)

# Replicando base de treino para medição de tempo de execução de algoritmos
for i in range(6):
  x_treino = pd.concat([x_treino, x_treino])
  y_treino = pd.concat([y_treino, y_treino])

print('Tamanho da base de treino:', y_treino.shape)

for idx, clf in enumerate(classifiers):
  inicio = time.time()

  if classifier_names[idx]=='CatBoost':
    mod = clf.fit(x_treino, y_treino, silent= True)
  else:
    mod = clf.fit(x_treino, y_treino)

  y_prev = mod.predict(x_teste)
  acc = accuracy_score(y_teste, y_prev)
  fim = time.time()

  print(classifier_names[idx], '| Acc=',acc,'| Tempo para execução:', fim-inicio, 'segundos')
```

```
Tamanho da base de treino: (38336,)
GBC | Acc= 0.7636986301369864 | Tempo para execução: 1.3645613193511963 segundos
XGBoost | Acc= 0.7705479452054794 | Tempo para execução: 1.254347801208496 segundos
LightGBM | Acc= 0.7602739726027398 | Tempo para execução: 0.2991676330566406 segundos
CatBoost | Acc= 0.7328767123287672 | Tempo para execução: 6.484435558319092 segundos
```

Figura 14 – Execução dos 4 algoritmos e comparação entre eles, com aumento de base
Fonte: Elaborada pelo autor (2022)

Cada um destes algoritmos tem uma série de parâmetros para serem ajustados. Não é uma tarefa simples equalizar estes parâmetros – a exemplo do que foi feito na Figura “Comparação da função perda para XGBoost, LightGBM e CatBoost” –, mas vamos tentar um pequeno ajuste de parâmetros no CatBoost. Seguiremos os mesmos parâmetros utilizados no artigo base dessa figura.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```python
from catboost.core import CatBoostClassifier
import time

classifiers = [
    GradientBoostingClassifier(),
    XGBClassifier(),
    lgb.LGBMClassifier(),
    ctb.CatBoostClassifier()
]
classifier_names = ['GBC', 'XGBoost', 'LightGBM', 'CatBoost']

x_treino, x_teste, y_treino, y_teste = train_test_split(df[['female', 'class_1','class_2', 'SibSp', 'Parch', 'AgeFilledNa']]
                                                        , df['Survived'],test_size=0.3266, random_state=1)

# Replicando base de treino para medição de tempo de execução de algoritmos
for i in range(6):
  x_treino = pd.concat([x_treino, x_treino])
  y_treino = pd.concat([y_treino, y_treino])

for idx, clf in enumerate(classifiers):
  inicio = time.time()

  if classifier_names[idx]=='CatBoost':
    clf = CatBoostClassifier(iterations=20, learning_rate=0.15,depth=10)
    mod = clf.fit(x_treino, y_treino, silent= True)
  else:
    mod = clf.fit(x_treino, y_treino)

  y_prev = mod.predict(x_teste)
  acc = accuracy_score(y_teste, y_prev)
  fim = time.time()

  print(classifier_names[idx], '| Acc=',acc,'| Tempo para execução:', fim-inicio, 'segundos')
```

```
GBC | Acc= 0.7636986301369864 | Tempo para execução: 1.3642094135284424 segundos
XGBoost | Acc= 0.7705479452054794 | Tempo para execução: 1.2574260234832764 segundos
LightGBM | Acc= 0.7602739726027398 | Tempo para execução: 0.2980821132659912 segundos
CatBoost | Acc= 0.7705479452054794 | Tempo para execução: 0.16127562522888184 segundos
```

Figura 15 – Execução dos 4 algoritmos e comparação entre eles, com ajustes de parâmetros no CatBoost
Fonte: Elaborada pelo autor (2022)

Conseguimos um resultado melhor e tempo de processamento menor. Mas será que para este dataset, a distribuição de dados na base de treino e teste importa? Vamos realizar, para finalizar, um laço desde a etapa de split de bases para avaliar justamente isto. Veja, na Figura “Implementação do lação para avaliar diferentes distribuições de bases treino-teste”, o Código ajustado com o laço.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```python
from catboost.core import CatBoostClassifier
import time

classifiers = [
    GradientBoostingClassifier(),
    XGBClassifier(),
    lgb.LGBMClassifier(),
    ctb.CatBoostClassifier()
]
classifier_names = ['GBC', 'XGBoost', 'LightGBM', 'CatBoost']

for iRandomState in range(5):

  x_treino, x_teste, y_treino, y_teste = train_test_split(df[['female', 'class_1','class_2', 'SibSp', 'Parch', 'AgeFilledNa']]
                                                          , df['Survived'],test_size=0.3266, random_state=iRandomState)

  # Replicando base de treino para medição de tempo de execução de algoritmos
  for i in range(6):
    x_treino = pd.concat([x_treino, x_treino])
    y_treino = pd.concat([y_treino, y_treino])

  for idx, clf in enumerate(classifiers):
    inicio = time.time()

    if classifier_names[idx]=='CatBoost':
      clf = CatBoostClassifier(iterations=20, learning_rate=0.15,depth=10)
      mod = clf.fit(x_treino, y_treino, silent= True)
    else:
      mod = clf.fit(x_treino, y_treino)

    y_prev = mod.predict(x_teste)
    acc = accuracy_score(y_teste, y_prev)
    fim = time.time()

    print(classifier_names[idx], '| Acc=',acc,'| Tempo para execução:', fim-inicio, 'segundos')
```

Figura 16 – Implementação do lação para avaliar diferentes distribuições de bases treino-teste
Fonte: Elaborada pelo autor (2022)

E por fim, na Figura “Resultados das 5 iterações do lação, com diferentes bases treino-teste”, os resultados, deixando claro que há bastante variação, pois em algumas situações um algoritmo é melhor que outro, porém, com alternância. Estes resultados não são completamente aleatórios, mas demonstram a importância de avaliar criteriosamente os parâmetros e conhecer mais detalhes de cada algoritmo.

```
GBC | Acc= 0.8116438356164384 | Tempo para execução: 1.3730709552764893 segundos
XGBoost | Acc= 0.8047945205479452 | Tempo para execução: 1.2530746459960938 segundos
LightGBM | Acc= 0.7773972602739726 | Tempo para execução: 0.2980766296386719 segundos
CatBoost | Acc= 0.8184931506849316 | Tempo para execução: 0.16159725189208984 segundos
GBC | Acc= 0.7602739726027398 | Tempo para execução: 1.3432857990264893 segundos
XGBoost | Acc= 0.7705479452054794 | Tempo para execução: 1.25520658493042 segundos
LightGBM | Acc= 0.7602739726027398 | Tempo para execução: 0.29300403594970703 segundos
CatBoost | Acc= 0.7705479452054794 | Tempo para execução: 0.15827441215515137 segundos
GBC | Acc= 0.797945205479452 | Tempo para execução: 1.363389492034912 segundos
XGBoost | Acc= 0.8013698630136986 | Tempo para execução: 1.2402095794677734 segundos
LightGBM | Acc= 0.773972602739726 | Tempo para execução: 0.29149818420410156 segundos
CatBoost | Acc= 0.7945205479452054 | Tempo para execução: 0.16299152374267578 segundos
GBC | Acc= 0.7876712328767124 | Tempo para execução: 1.3616306781768799 segundos
XGBoost | Acc= 0.7808219178082192 | Tempo para execução: 1.2028133869171143 segundos
LightGBM | Acc= 0.797945205479452 | Tempo para execução: 0.2976956367492676 segundos
CatBoost | Acc= 0.8013698630136986 | Tempo para execução: 0.15016460418701172 segundos
GBC | Acc= 0.839041095890411 | Tempo para execução: 1.383047342300415 segundos
XGBoost | Acc= 0.8424657534246576 | Tempo para execução: 1.2244505882263184 segundos
LightGBM | Acc= 0.8047945205479452 | Tempo para execução: 0.286837100982666 segundos
CatBoost | Acc= 0.821917808219178 | Tempo para execução: 0.15635228157043457 segundos
```

Figura 17 – Resultados das 5 iterações do lação, com diferentes bases treino-teste
Fonte: Elaborada pelo autor (2022)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

REFERÊNCIAS

B. EFRON. Bootstrap Methods: Another Look at the Jackknife. **Ann. Statist.** 7 (1) 1-26, jan. 1979. Disponível em: https://doi.org/10.1214/aos/1176344552. Acesso em: 19 set. 2022.

BENTÉJAC, C.; CSÖRGŐ, A.; MARTÍNEZ-MUÑOZ, G. A comparative analysis of gradient boosting algorithms. **Artif Intell Rev**, 54, 1937–1967 (2021). Disponível em: https://doi.org/10.1007/s10462-020-09896-5. Acesso em: 19 set. 2022.

CHEN, T., GUESTRIN, C. **XGBoost: A Scalable Tree Boosting System**. KDD '16: Proceedings of the 22nd ACM SIGKDD **International Conference on Knowledge Discovery and Data Mining**. p. 785–794, aug. 2016. Disponível em: https://doi.org/10.1145/2939672.2939785. Acesso em: 19 set. 2022.

DOROGUSH, A.V., ERSHOV, V., GULIN, A. **CatBoost: gradient boosting with categorical features support**. arXiv. dec. 2018. Disponível em: https://doi.org/10.48550/arXiv.1810.11363. Acesso em: 19 set. 2022.

KE, G., et al. **LightGBM: A Highly Efficient Gradient Boosting Decision Tree.** Advances in Neural Information Processing Systems 30 (NIP 2017). Disponível em: https://papers.nips.cc/paper/2017/file/6449f44a102fde848669bdd9eb6b76fa-Paper.pdf. Acesso em: 19 set. 2022.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com